

- [背景知识](#)
 - [Word Embedding Vector](#)
 - [nn.Linear](#)
- [二, Transformer 输入](#)
 - [1.1, 单词 Embedding](#)
 - [1.2, 位置 Embedding](#)
 - [1.1, TransformerEmbedding 层实现](#)
- [二, Transformer 整体结构](#)
- [三, Multi-Head Attention 结构](#)
 - [3.1, Self-Attention 结构](#)
 - [3.2, Self-Attention 实现](#)
 - [3.3, Multi-Head Attention](#)
 - [3.4, Multi-Head Attention 实现](#)
- [四, Encoder 结构](#)
 - [4.1, Add & Norm](#)
 - [4.2, Feed Forward](#)
 - [4.3, Encoder 结构的实现](#)
- [五, Decoder 结构](#)
- [六, Transformer 总结](#)
 - [6.1, Transformer 完整代码实现](#)
- [参考资料](#)

背景知识

Word Embedding Vector

Word Embedding Vector 是将单词映射到向量空间的一种方式，它可以将单词的语义信息转化为向量形式，并且可以在向量空间中计算单词之间的相似性，简单理解就是，将单词从原始文本格式转换为神经网络可以理解和处理的数字格式的一种方式。在深度学习中，我们通常使用 Word2Vec、GloVe 或 FastText 等算法来生成单词的嵌入向量。

下面是一个简单的 Python 示例，用于使用 PyTorch 生成单词的嵌入向量：

```
1 import torch
2 import torch.nn as nn
3 from torchtext.vocab import Vocab
4
5 from collections import Counter
6
7 text = "hello world hello" # 假设我们有一个文本
8 counter = Counter(text.split()) # 统计每个单词的出现次数
9 vocab = Vocab(counter) # 创建词汇表
10
11 # 构建嵌入层
```

```

12 vocab_size = 1000
13 embedding_dim = 100
14 embedding_layer = nn.Embedding(vocab_size, embedding_dim)
15
16 # 假设我们有一个输入单词列表，每个单词都是从词汇表中随机选择的
17 input_words = ["hello", "world", "this", "is", "a", "test"]
18
19 # 将单词转换为索引列表
20 word_indexes = [vocab.index(word) for word in input_words]
21
22 # 将索引列表转换为PyTorch张量
23 word_indexes_tensor = torch.LongTensor(word_indexes)
24
25 # 将索引列表输入嵌入层以获取嵌入向量
26 word_embeddings = embedding_layer(word_indexes_tensor)
27
28 # 输出嵌入向量
29 print(word_embeddings)

```

1. 首先通过 `vocab` 和输入文本字符串创建词汇表 `vocab`，并通过 `nn.Embedding` 创建了一个大小为 1000×100 的嵌入层 `embedding_layer`，表示词汇表中有 1000 个单词，每个单词用一个 100 维的向量表示。
2. 然后，我们创建一个输入单词列表 `input_words`，并将每个单词转换为词汇表中对应的索引，并将索引列表转换为 PyTorch 张量。
3. 最后将张量 `word_indexes_tensor` 输入到嵌入层中，以获取每个单词的 100 维嵌入向量。

值得注意的是，嵌入向量的大小和维度通常需要根据任务进行调整。通常，词汇表越大，嵌入向量的维度就越高。此外，嵌入向量的大小通常需要与模型中的其他参数相匹配，例如隐藏层的大小和注意力头的数量。

nn.Linear

在 PyTorch 中，`nn.Linear` 是一个模块，它实现了一个全连接层，它将输入张量的每个元素都乘以一个可学习的权重矩阵，并加上一个可学习的偏置向量，最后输出一个新的张量。全连接层的数学表达式为：

$$\text{output} = \text{input} \times \text{weight}^T + \text{bias}$$

`nn.Linear` 的构造函数如下：

```

1 nn.Linear(in_features: int, out_features: int, bias: bool = True)

```

各参数解释如下：

- `in_features` 表示输入张量的特征数，也就是输入张量的最后一维的大小，
- `out_features` 表示输出张量的特征数，也就是输出张量的最后一维的大小，
- `bias` 是一个布尔值，表示是否添加偏置向量。如果 `bias` 为 `False`，则不会添加偏置向量。

下面是一个示例代码，可直接运行，其创建了一个输入张量，并通过层进行维度的线性变换。

```

1 import torch.nn as nn

```

```

2 import torch
3 import time
4
5 # 创建一个输入张量
6 input_tensor = torch.randn(10, 20)
7
8 # 创建一个 Linear 层对象, 将输入维度从 20 变换到 30
9 linear_layer = nn.Linear(20, 30)
10
11 # 将输入张量传递给 Linear 层进行变换
12 start_time = time.time()
13 output_tensor = linear_layer(input_tensor)
14 end_time = time.time()
15
16 # 打印输出张量的形状
17 print(f"Output shape: {output_tensor.shape}") # Output shape: torch.Size([10, 30])
18 print(f"Time taken: {end_time - start_time:.6f} seconds") # Time taken: 0.002543
    seconds

```

一，Transformer 输入

Transformer 中单词的输入表示 $\mathbf{x}$ 由单词 **Embedding** 和位置 **Embedding**（Positional Encoding）相加得到，通常定义为 TransformerEmbedding 层，其代码实现如下所示：

1.1，单词 Embedding

单词的 Embedding 有很多方式可以获取，例如可以采用 Word2Vec、Glove 等算法预训练得到，也可以在 Transformer 中训练得到。

1.2，位置 Embedding

Transformer 中除了单词的 Embedding，还需要使用位置 Embedding 表示单词出现在句子中的位置。因为 Transformer 不采用 RNN 的结构，而是使用全局信息，不能利用单词的顺序信息，而这部分信息对于 NLP 来说非常重要。所以 Transformer 中使用位置 Embedding 保存单词在序列中的相对或绝对位置。

位置 Embedding 用 PE 表示，PE 的维度与单词 Embedding 是一样的。PE 可以通过训练得到，也可以使用某种公式计算得到。在 Transformer 中采用了后者，计算公式如下：

$$\begin{aligned}
 PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{model}}) \\
 PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{model}})
 \end{aligned} \tag{3}$$

其中，pos 表示单词在句子中的位置，d 表示 PE 的维度 (与词 Embedding 一样)，2i 表示偶数的维度，2i+1 表示奇数维度 (即 $2i \leq d, 2i+1 \leq d$)。

1.1, TransformerEmbedding 层实现

```
1 class PositionalEncoding(nn.Module):
2     """
3     compute sinusoid encoding.
4     """
5     def __init__(self, d_model, max_len, device):
6         """
7         constructor of sinusoid encoding class
8
9         :param d_model: dimension of model
10        :param max_len: max sequence length
11        :param device: hardware device setting
12        """
13        super(PositionalEncoding, self).__init__()
14
15        # same size with input matrix (for adding with input matrix)
16        self.encoding = torch.zeros(max_len, d_model, device=device)
17        self.encoding.requires_grad = False # we don't need to compute gradient
18
19        pos = torch.arange(0, max_len, device=device)
20        pos = pos.float().unsqueeze(dim=1)
21        # 1D => 2D unsqueeze to represent word's position
22
23        _2i = torch.arange(0, d_model, step=2, device=device).float()
24        # 'i' means index of d_model (e.g. embedding size = 50, 'i' = [0,50])
25        # "step=2" means 'i' multiplied with two (same with 2 * i)
26
27        self.encoding[:, 0::2] = torch.sin(pos / (10000 ** (_2i / d_model)))
28        self.encoding[:, 1::2] = torch.cos(pos / (10000 ** (_2i / d_model)))
29        # compute positional encoding to consider positional information of words
30
31    def forward(self, x):
32        # self.encoding
33        # [max_len = 512, d_model = 512]
34
35        batch_size, seq_len = x.size()
36        # [batch_size = 128, seq_len = 30]
37
38        return self.encoding[:seq_len, :]
39        # [seq_len = 30, d_model = 512]
40        # it will add with tok_emb : [128, 30, 512]
41
42 class TokenEmbedding(nn.Embedding):
43     """
44     Token Embedding using torch.nn
45     they will dense representation of word using weighted matrix
46     """
```

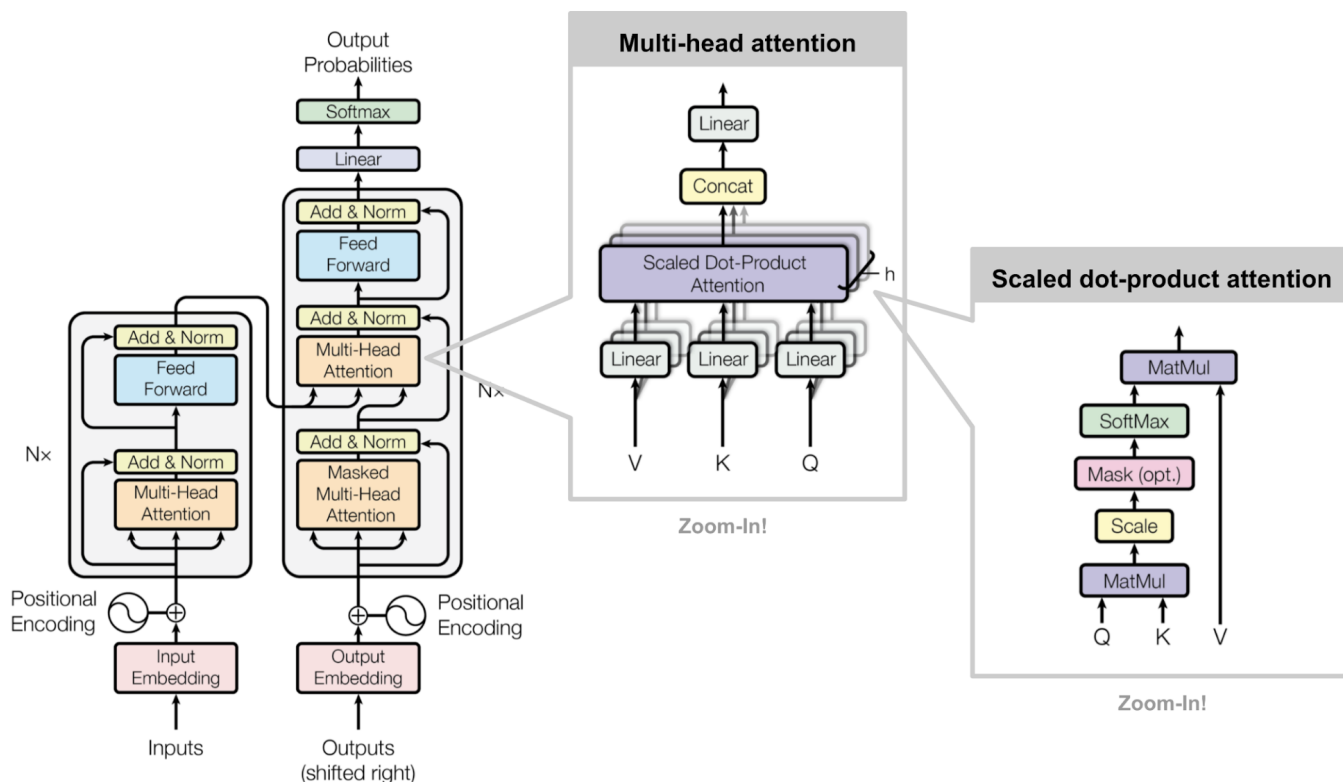
```

47
48     def __init__(self, vocab_size, d_model):
49         """
50         class for token embedding that included positional information
51         :param vocab_size: size of vocabulary
52         :param d_model: dimensions of model
53         """
54         super(TokenEmbedding, self).__init__(vocab_size, d_model, padding_idx=1)
55
56 class TransformerEmbedding(nn.Module):
57     """
58     token embedding + positional encoding (sinusoid)
59     positional encoding can give positional information to network
60     """
61
62     def __init__(self, vocab_size, max_len, d_model, drop_prob, device):
63         """
64         class for word embedding that included positional information
65         :param vocab_size: size of vocabulary
66         :param d_model: dimensions of model
67         """
68         super(TransformerEmbedding, self).__init__()
69         self.tok_emb = TokenEmbedding(vocab_size, d_model)
70         self.pos_emb = PositionalEncoding(d_model, max_len, device)
71         self.drop_out = nn.Dropout(p=drop_prob)
72
73     def forward(self, x):
74         tok_emb = self.tok_emb(x)
75         pos_emb = self.pos_emb(x)
76         return self.drop_out(tok_emb + pos_emb)

```

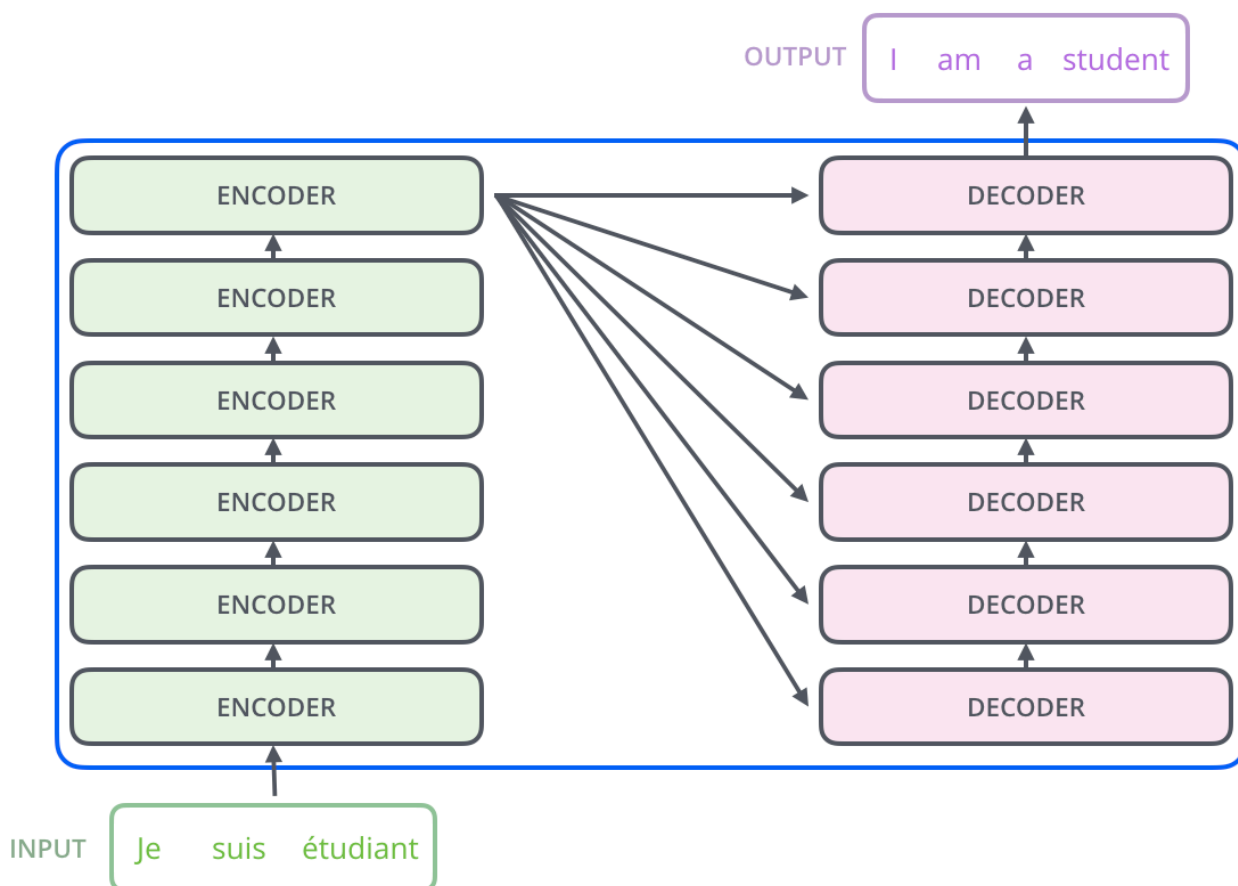
二，Transformer 整体结构

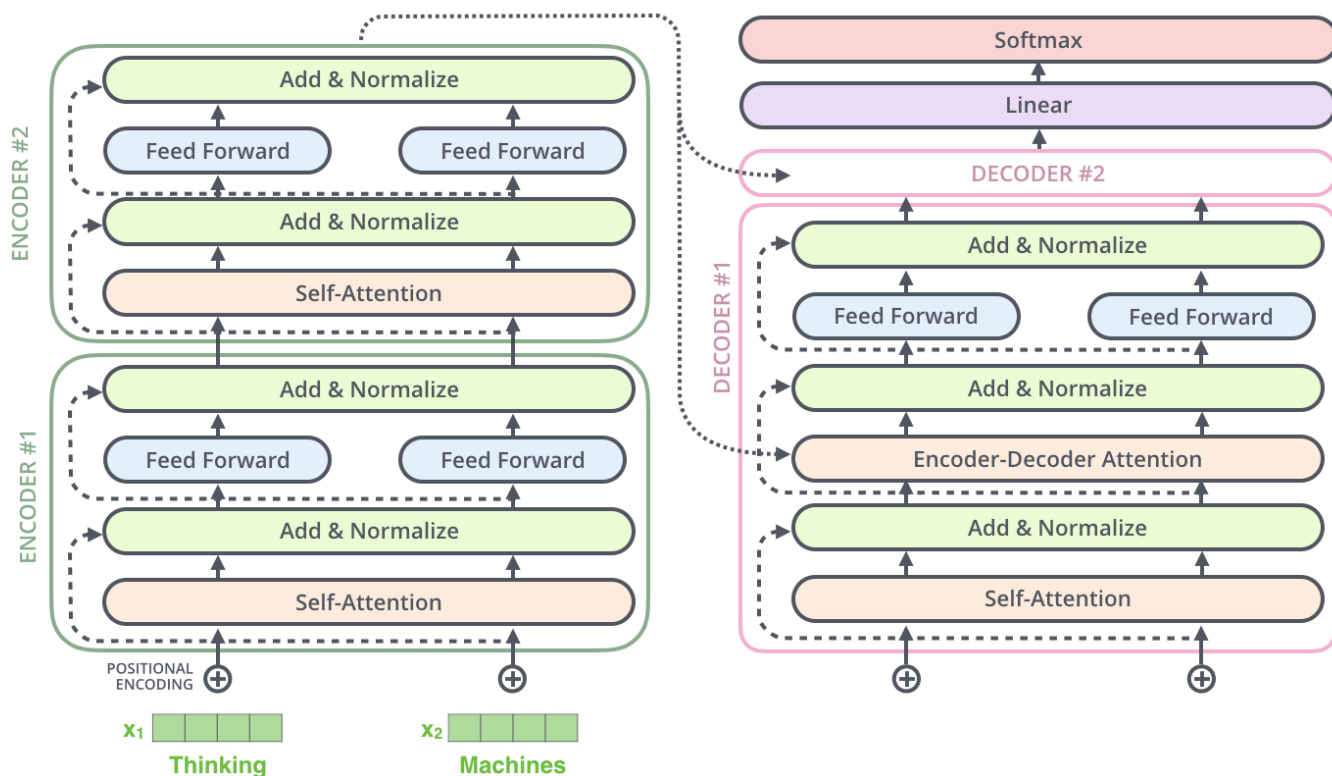
论文中给出用于中英文翻译任务的 `Transformer` 整体架构如下图所示：



可以看出 Transformer 架构由 **Encoder** 和 **Decoder** 两个部分组成：其中 Encoder 和 Decoder 都是由 $N=6$ 个相同的层堆叠而成。**Multi-Head Attention** 结构是 Transformer 架构的核心结构，其由多个 **Self-Attention** 组成的。

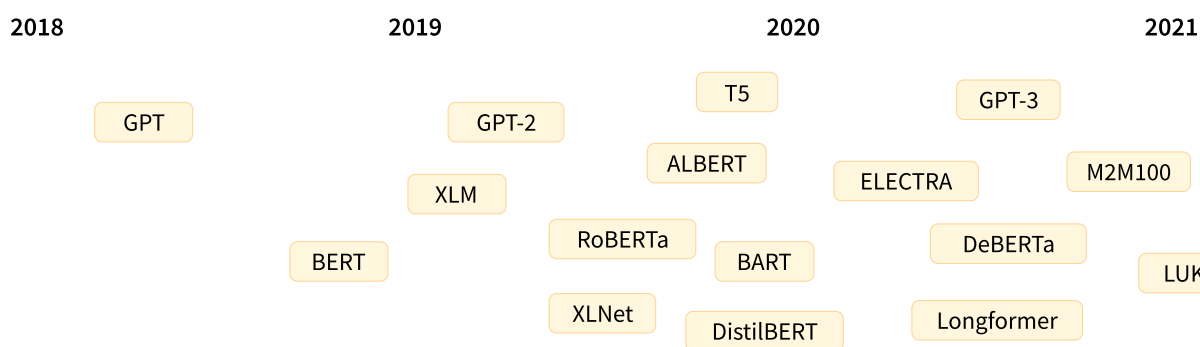
Transformer 架构更详细的可视化图如下所示：





2.1, Transformer 发展史

以下是 Transformer 模型（简短）历史中的一些关键节点：



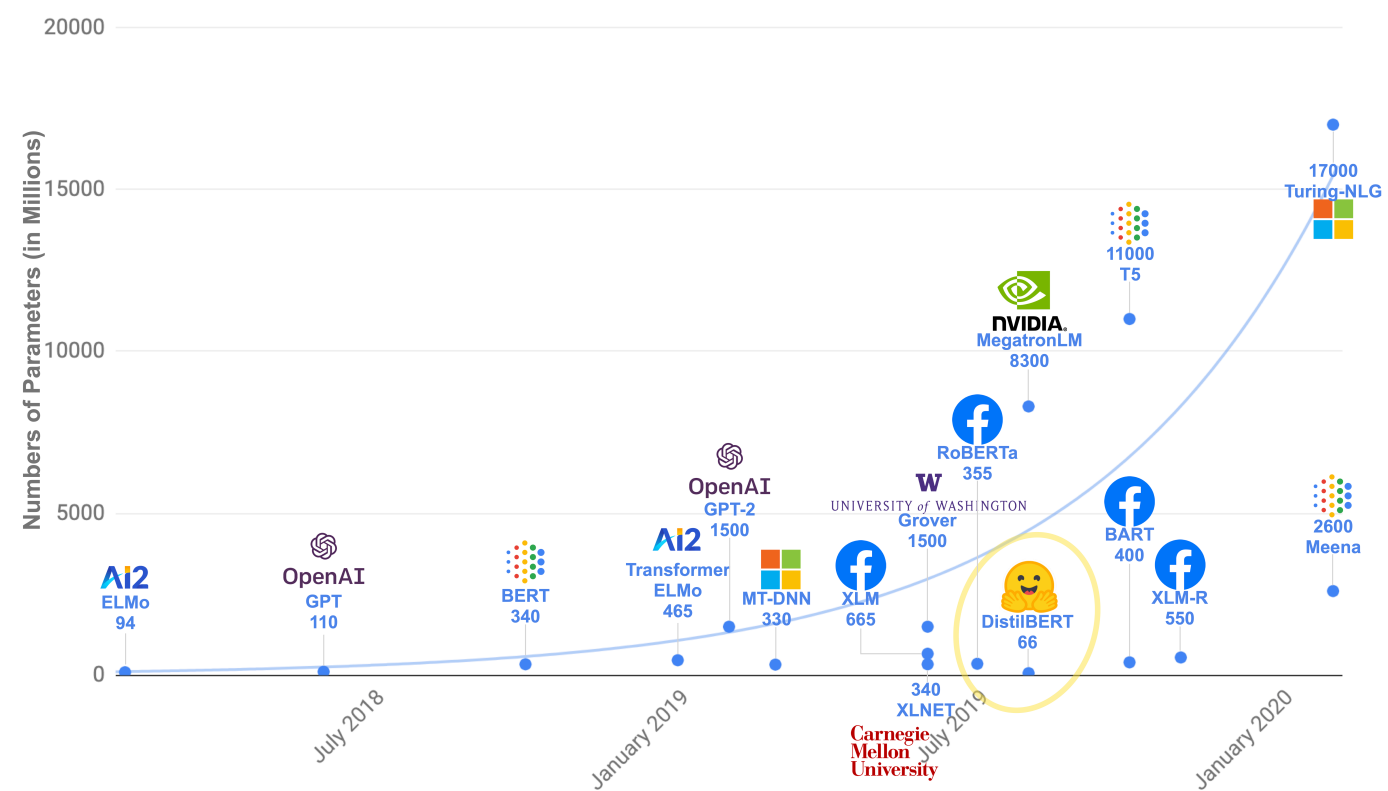
[Transformer 架构](#) 于 2017 年 6 月推出。原本研究的重点是翻译任务。随后推出了几个有影响力的模型，包括

- **2018 年 6 月:** [GPT](#), 第一个预训练的 Transformer 模型，用于各种 NLP 任务并获得极好的结果
- **2018 年 10 月:** [BERT](#), 另一个大型预训练模型，该模型旨在生成更好的句子摘要（下一章将详细介绍！）
- **2019 年 2 月:** [GPT-2](#), GPT 的改进（并且更大）版本，由于道德问题没有立即公开发布
- **2019 年 10 月:** [DistilBERT](#), BERT 的提炼版本，速度提高 60%，内存减轻 40%，但仍保留 BERT 97% 的性能
- **2019 年 10 月:** [BART](#) 和 [T5](#), 两个使用与原始 Transformer 模型相同架构的大型预训练模型（第一个这样做）
- **2020 年 5 月:** [GPT-3](#), GPT-2 的更大版本，无需微调即可在各种任务上表现良好（称为零样本学习）

这个列表并不全面，只是为了突出一些不同类型的 Transformer 模型。大体上，它们可以分为三类：

- GPT-like (也被称作自回归 Transformer 模型)
- BERT-like (也被称作自动编码 Transformer 模型)
- BART/T5-like (也被称作序列到序列的 Transformer 模型)

Transformer 是大模型，除了一些特例（如 DistilBERT）外，实现更好性能的一般策略是增加模型的大小以及预训练的数据量。



模型	发布时间	参数量	预训练数据量
GPT	2018 年 6 月	1.17 亿	约 5GB
GPT-2	2019 年 2 月	15 亿	40GB
GPT-3	2020 年 5 月	1,750 亿	45TB

三，Multi-Head Attention 结构

Encoder 和 Decoder 结构中公共的 layer 之一是 Multi-Head Attention，其是由多个 Self-Attention 并行组成的。Encoder block 只包含一个 Multi-Head Attention，而 Decoder block 包含两个 Multi-Head Attention (其中有一个用到 Masked)。

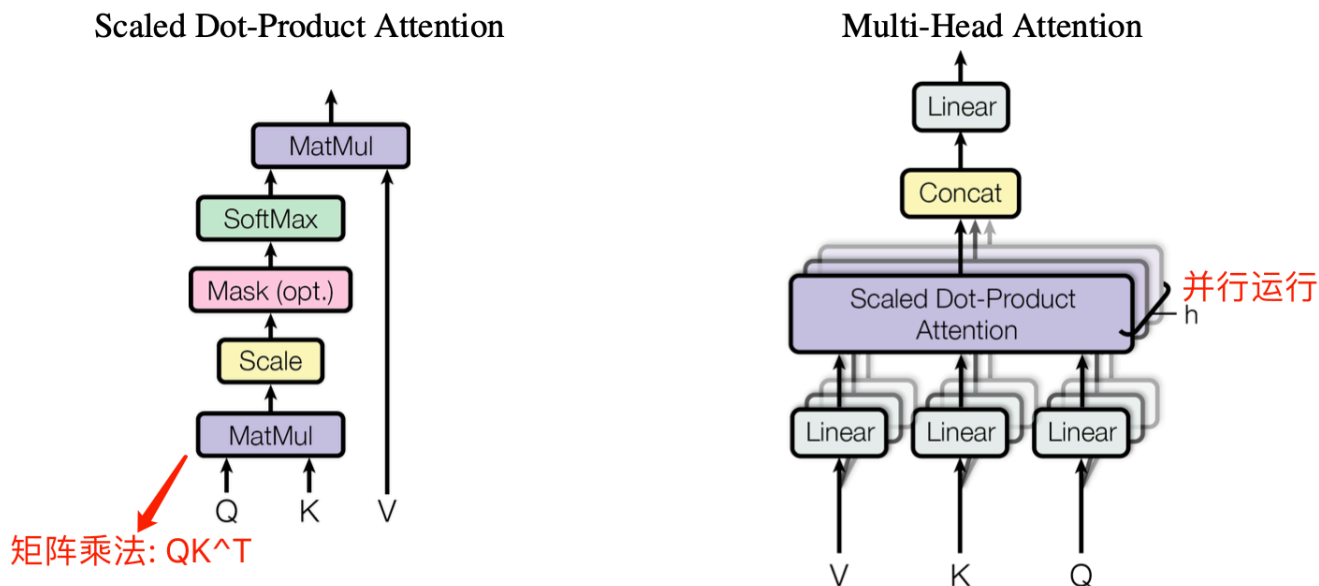
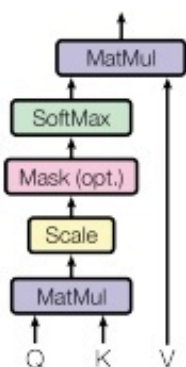


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

3.1, Self-Attention 结构

Self-Attention 中文翻译为自注意力机制，论文中叫作 **Scale Dot Product Attention**，它是 Transformer 架构的核心，其结构如下图所示：

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

那么重点来了，第一个问题：Self-Attention 结构的最初输入 **Q(查询)**, **K(键值)**, **V(值)** 这三个矩阵怎么理解呢？其代表什么，通过什么计算而来？

在 Self-Attention 中， Q 、 K 、 V 是在同一个输入（比如序列中的一个单词）上计算得到的三个向量。具体来说，我们可以通过对原始输入词的 **embedding** 进行线性变换（比如使用一个全连接层），来得到 Q 、 K 、 V 。这三个向量的维度通常都是一样的，取决于模型设计时的决策。

第二个问题：Self-Attention 结构怎么理解， Q 、 K 、 V 的作用是什么？这三个矩阵又怎么计算得到最后的输出？

在计算 Self-Attention 时，Q、K、V 被用来计算注意力分数，即用于表示当前位置和其他位置之间的关系。注意力分数可以通过 Q 和 K 的点积来计算，然后将分数除以 8，再经过一个 softmax 归一化处理，得到每个位置的权重。然后用这些权重来加权计算 V 的加权和，即得到当前位置的输出。

将分数除以 8 的操作，对应图中的 Scale 层，这个参数 8 是 K 向量维度 64 的平方根结果。

3.2, Self-Attention 实现

文章的 Self-Attention 层和论文中的 ScaleDotProductAttention 层意义是一样的。

输入序列单词的 Embedding Vector 经过线性变换 (Linear 层) 得到 Q、K、V 三个向量，并将它们作为 Self-Attention 层的输入。假设输入序列的长度为 seq_len，则 Q、K 和 V 的形状为 (seq_len, d_k)，其中，d_k 表示每个词或向量的维度，也是 Q、K 矩阵的列数。在论文中，输入给 Self-Attention 层的 Q、K、V 的向量维度是 64，Embedding Vector 和 Encoder-Decoder 模块输入输出的维度都是 512。

Embedding Vector 的大小是我们设置的超参数——基本上它就是训练数据集中最长句子的长度。

Self-Attention 层的计算过程用数学公式可表达为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

以下是一个示例代码，它创建了一个 ScaleDotProductAttention 层，并将 Q、K、V 三个张量传递给它进行计算：

```
1 class ScaleDotProductAttention(nn.Module):
2     def __init__(self, ):
3         super(ScaleDotProductAttention, self).__init__()
4         self.softmax = nn.Softmax(dim = -1)
5
6     def forward(self, Q, K, V, mask=None):
7         K_T = K.transpose(-1, -2) # 计算矩阵 K 的转置
8         d_k = Q.size(-1)
9         # 1, 计算 Q, K^T 矩阵的点积，再除以 sqrt(d_k) 得到注意力分数矩阵
10        scores = torch.matmul(Q, K_T) / math.sqrt(d_k)
11        # 2, 如果有掩码，则将注意力分数矩阵中对应掩码位置的值设为负无穷大
12        if mask is not None:
13            scores = scores.masked_fill(mask == 0, -1e9)
14        # 3, 对注意力分数矩阵按照最后一个维度进行 softmax 操作，得到注意力权重矩阵，值范围为
[0, 1]
15        attn_weights = self.softmax(scores)
16        # 4, 将注意力权重矩阵乘以 v，得到最终的输出矩阵
17        output = torch.matmul(attn_weights, V)
18
19        return output, attn_weights
20
21 # 创建 Q、K、V 三个张量
22 Q = torch.randn(5, 10, 64) # (batch_size, sequence_length, d_k)
23 K = torch.randn(5, 10, 64) # (batch_size, sequence_length, d_k)
24 V = torch.randn(5, 10, 64) # (batch_size, sequence_length, d_k)
25
26 # 创建 ScaleDotProductAttention 层
```

```

27 attention = ScaleDotProductAttention()
28
29 # 将 Q、K、V 三个张量传递给 ScaleDotProductAttention 层进行计算
30 output, attn_weights = attention(Q, K, V)
31
32 # 打印输出矩阵和注意力权重矩阵的形状
33 print(f"ScaleDotProductAttention output shape: {output.shape}") # torch.Size([5, 10, 64])
34 print(f"attn_weights shape: {attn_weights.shape}") # torch.Size([5, 10, 10])

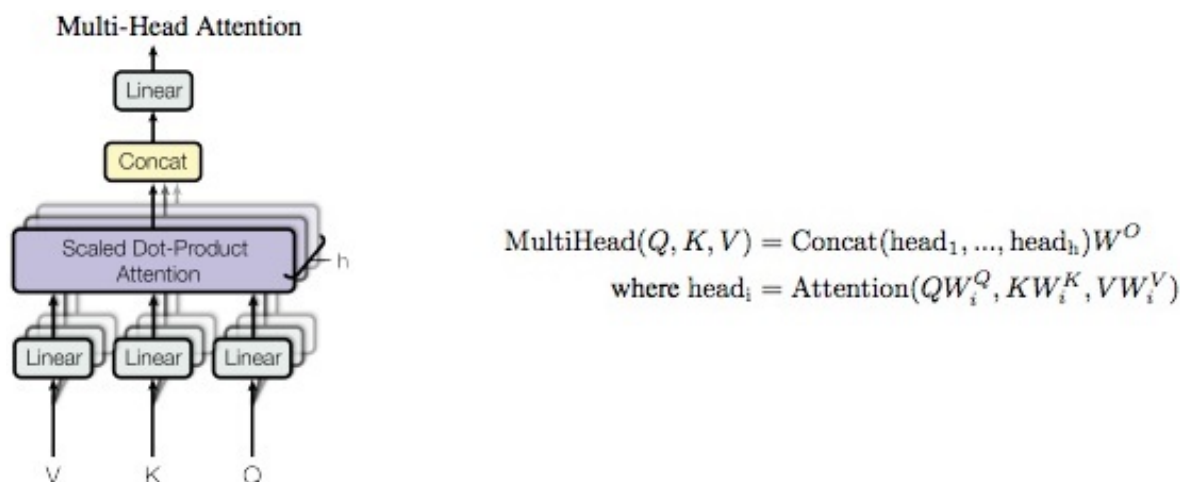
```

3.3, Multi-Head Attention

Multi-Head Attention (MHA) 是基于 Self-Attention (SA) 的一种变体。MHA 在 SA 的基础上引入了“多头”机制，将输入拆分为多个子空间，每个子空间分别执行 SA，最后将多个子空间的输出拼接在一起并进行线性变换，从而得到最终的输出。

对于 MHA，之所以需要对 Q、K、V 进行多头 (head) 划分，其目的是为了增强模型对不同信息的关注。具体来说，多组 Q、K、V 分别计算 Self-Attention，每个头自然就会有独立的 Q、K、V 参数，从而让模型同时关注多个不同的信息，这有些类似 CNN 架构模型的多通道机制。

下图是论文中 Multi-Head Attention 的结构图。



从图中可以看出，MHA 结构的计算过程可总结为下述步骤：

1. 将输入 Q、K、V 张量进行线性变换 (Linear 层)，输出张量尺寸为 [batch_size, seq_len, d_model];
2. 将前面步骤输出的张量，按照头的数量 (n_head) 拆分为 n_head 子张量，其尺寸为 [batch_size, n_head, seq_len, d_model//n_head];
3. 每个子张量并行计算注意力分数，即执行 dot-product attention 层，输出张量尺寸为 [batch_size, n_head, seq_len, d_model//n_head];
4. 将这些子张量进行拼接 concat，并经过线性变换得到最终的输出张量，尺寸为 [batch_size, seq_len, d_model]。

总结：因为 GPU 的并行计算特性，步骤2中的张量拆分和步骤4中的张量拼接，其实都是通过 review 算子来实现的。同时，也能发现 SA 和 MHA 模块的输入输出矩阵维度都是一样的。

3.4, Multi-Head Attention 实现

Multi-Head Attention 层的输入同样也是三个张量：查询（Query）、键（Key）和值（Value），其计算过程用数学公式可表达为：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (4)$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

一般用 `d_model` 表示输入嵌入向量的维度，`n_head` 表示分割成多少个头，因此，`d_model//n_head` 自然表示每个头的输入和输出维度，在论文中 `d_model = 512`，`n_head = 8`，`d_model//n_head = 64`。值得注意的是，由于每个头的维数减少，总计算成本与具有全维的单头注意力是相似的。

Multi-Head Attention 层的 `Pytorch` 实现代码如下所示：

```
1 class MultiHeadAttention(nn.Module):
2     """Multi-Head Attention Layer
3     Args:
4         d_model: Dimensions of the input embedding vector, equal to input and
5         output dimensions of each head
6         n_head: number of heads, which is also the number of parallel attention
7         layers
8     """
9     def __init__(self, d_model, n_head):
10         super(MultiHeadAttention, self).__init__()
11         self.n_head = n_head
12         self.attention = ScaleDotProductAttention()
13         self.w_q = nn.Linear(d_model, d_model) # Q 线性变换层
14         self.w_k = nn.Linear(d_model, d_model) # K 线性变换层
15         self.w_v = nn.Linear(d_model, d_model) # V 线性变换层
16         self.fc = nn.Linear(d_model, d_model) # 输出线性变换层
17
18     def forward(self, q, k, v, mask=None):
19         # 1. dot product with weight matrices
20         q, k, v = self.w_q(q), self.w_k(k), self.w_v(v) # size is [batch_size,
21         seq_len, d_model]
22         # 2, split by number of heads(n_head) # size is [batch_size, n_head,
23         seq_len, d_model//n_head]
24         q, k, v = self.split(q), self.split(k), self.split(v)
25         # 3, compute attention
26         sa_output, attn_weights = self.attention(q, k, v, mask)
27         # 4, concat attention and linear transformation
28         concat_tensor = self.concat(sa_output)
29         mha_output = self.fc(concat_tensor)
30
31         return mha_output
32
33     def split(self, tensor):
34         """
35         split tensor by number of head(n_head)
```

```

32
33         :param tensor: [batch_size, seq_len, d_model]
34         :return: [batch_size, n_head, seq_len, d_model//n_head], 输出矩阵是四维的, 第
二个维度是 head 维度
35
36         # 将 Q、K、V 通过 reshape 函数拆分为 n_head 个头
37         batch_size, seq_len, _ = q.shape
38         q = q.reshape(batch_size, seq_len, n_head, d_model // n_head)
39         k = k.reshape(batch_size, seq_len, n_head, d_model // n_head)
40         v = v.reshape(batch_size, seq_len, n_head, d_model // n_head)
41         """
42
43         batch_size, seq_len, d_model = tensor.size()
44         d_tensor = d_model // self.n_head
45         split_tensor = tensor.view(batch_size, seq_len, self.n_head,
d_tensor).transpose(1, 2)
46         # it is similar with group convolution (split by number of heads)
47
48         return split_tensor
49
50     def concat(self, sa_output):
51         """ merge multiple heads back together
52
53         Args:
54             sa_output: [batch_size, n_head, seq_len, d_tensor]
55             return: [batch_size, seq_len, d_model]
56         """
57         batch_size, n_head, seq_len, d_tensor = sa_output.size()
58         d_model = n_head * d_tensor
59         concat_tensor = sa_output.transpose(1, 2).contiguous().view(batch_size,
seq_len, d_model)
60
61         return concat_tensor

```

四, Encoder 结构

Encoder 结构由 $N = 6$ 个相同的 encoder block 堆叠而成, 每一层 (layer) 主要有两个子层 (sub-layers): 第一个子层是多头注意力机制 (Multi-Head Attention), 第二个是简单的位置全连接前馈网络 (Positionwise Feed Forward)。

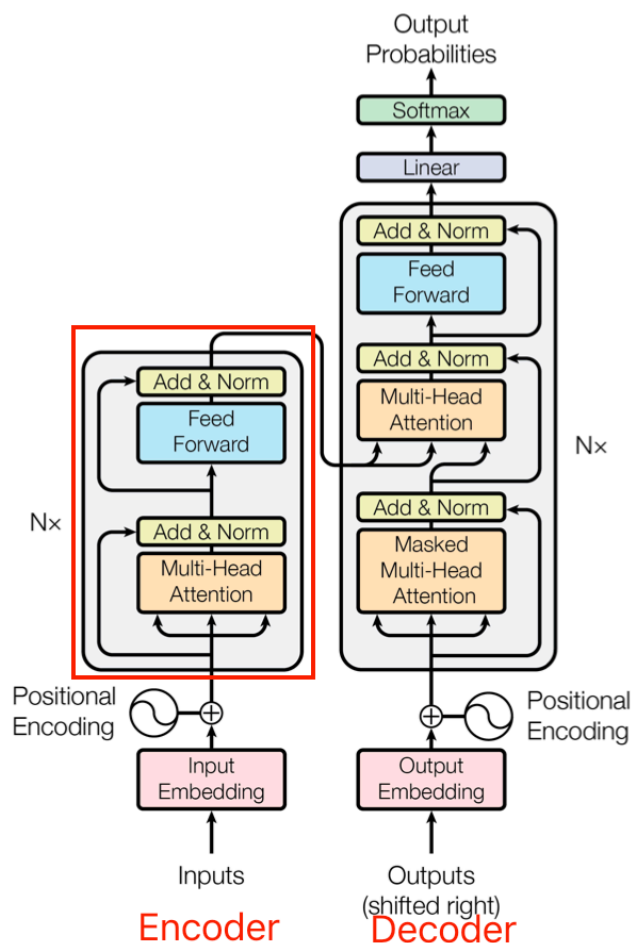


Figure 1: The Transformer - model architecture.

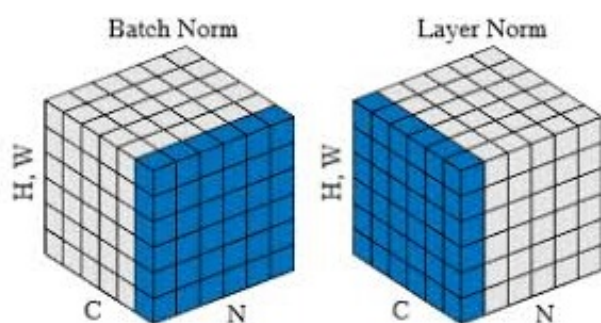
上图红色框框出的部分是 Encoder block，很明显其是由 Multi-Head Attention、Add&Norm、Feed Forward、Add & Norm 层组成的。另外在论文中 Encoder 组件由 $N = 6$ 个相同的 encoder block 堆叠而成，且 encoder block 输入矩阵和输出矩阵维度是一样的。

4.1, Add & Norm

Add & Norm 层由 Add 和 Norm 两部分组成。这里的 Add 指 $X + \text{MultiHeadAttention}(X)$ ，是一种残差连接。Norm 是 Layer Normalization。Add & Norm 层计算过程用数学公式可表达为：

$$\text{Layer Norm}(X + \text{MultiHeadAttention}(X))$$

Add 比较简单，这里重点讲下 Layer Norm 层。Layer Norm 是一种常用的神经网络归一化技术，可以使得模型训练更加稳定，收敛更快。它的主要作用是对每个样本在特征维度上进行归一化，减少了不同特征之间的依赖关系，提高了模型的泛化能力。Layer Norm 层的计算可视化如下图所示：



$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

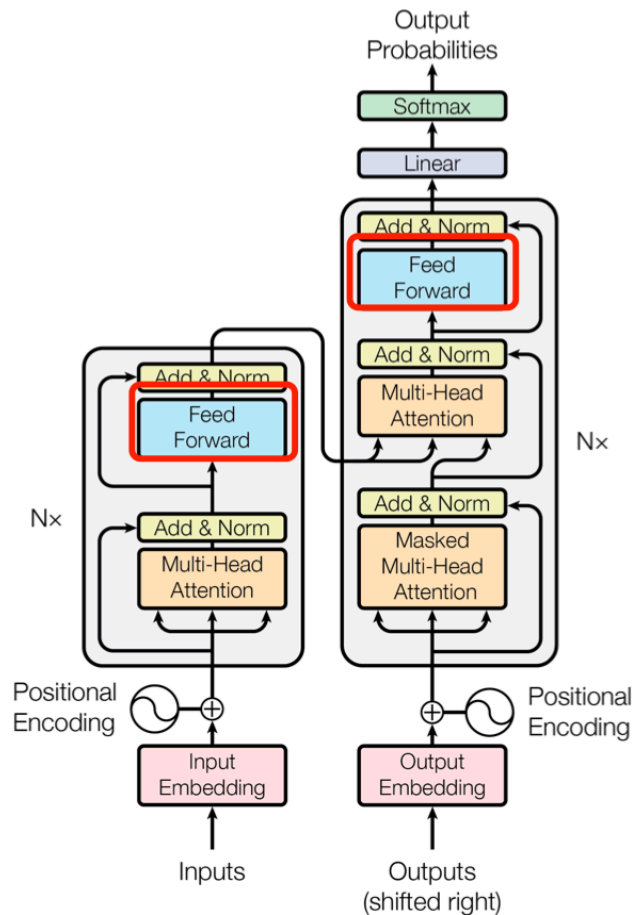
Layer Norm 层的 Pytorch 实现代码如下所示:

```

1  class LayerNorm(nn.Module):
2      def __init__(self, d_model, eps=1e-12):
3          super(LayerNorm, self).__init__()
4          self.gamma = nn.Parameter(torch.ones(d_model))
5          self.beta = nn.Parameter(torch.zeros(d_model))
6          self.eps = eps
7
8      def forward(self, x):
9          mean = x.mean(-1, keepdim=True) # '-1' means last dimension.
10         var = x.var(-1, keepdim=True)
11
12         out = (x - mean) / torch.sqrt(var + self.eps)
13         out = self.gamma * out + self.beta
14
15         return out
16
17     # NLP Example
18     batch, sentence_length, embedding_dim = 20, 5, 10
19     embedding = torch.randn(batch, sentence_length, embedding_dim)
20
21     # 1, Activate nn.LayerNorm module
22     layer_norm1 = nn.LayerNorm(embedding_dim)
23     pytorch_ln_out = layer_norm1(embedding)
24
25     # 2, Activate my nn.LayerNorm module
26     layer_norm2 = LayerNorm(embedding_dim)
27     my_ln_out = layer_norm2(embedding)
28
29     # 比较结果
30     print(torch.allclose(pytorch_ln_out, my_ln_out, rtol=0.1, atol=0.01)) # 输出 True

```


4.2, Feed Forward



Feed Forward 层全称是 Position-wise Feed-Forward Networks，其本质是一个**两层的全连接层**，第一层的激活函数为 Relu，第二层不使用激活函数，计算过程用数学公式可表达为：

$$\text{FFN}(X) = \max(0, XW_1 + b_1)W_2 + b_2$$

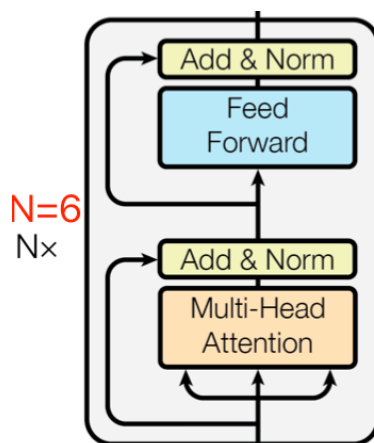
除了使用两个全连接层来完成线性变换，另外一种方式是使用 $\text{kernal_size} = 1$ 的两个 1×1 卷积层，输入输出维度不变，都是 512，中间维度是 2048。

PositionwiseFeedForward 层的 Pytorch 实现代码如下所示：

```
1 class PositionwiseFeedForward(nn.Module):
2     def __init__(self, d_model, d_diff, drop_prob=0.1):
3         super(PositionwiseFeedForward, self).__init__()
4         self.fc1 = nn.Linear(d_model, d_diff)
5         self.fc2 = nn.Linear(d_diff, d_model)
6         self.relu = nn.ReLU()
7         self.dropout = nn.Dropout(drop_prob)
8
9     def forward(self, x):
10         x = self.fc1(x)
11         x = self.relu(x)
12         x = self.dropout(x)
13         x = self.fc2(x)
14
15         return x
```


4.3, Encoder 结构的实现

基于前面 Multi-Head Attention, Feed Forward, Add & Norm 的内容我们可以完整的实现 Encoder 结构。



Encoder 组件的 Pytorch 实现代码如下所示:

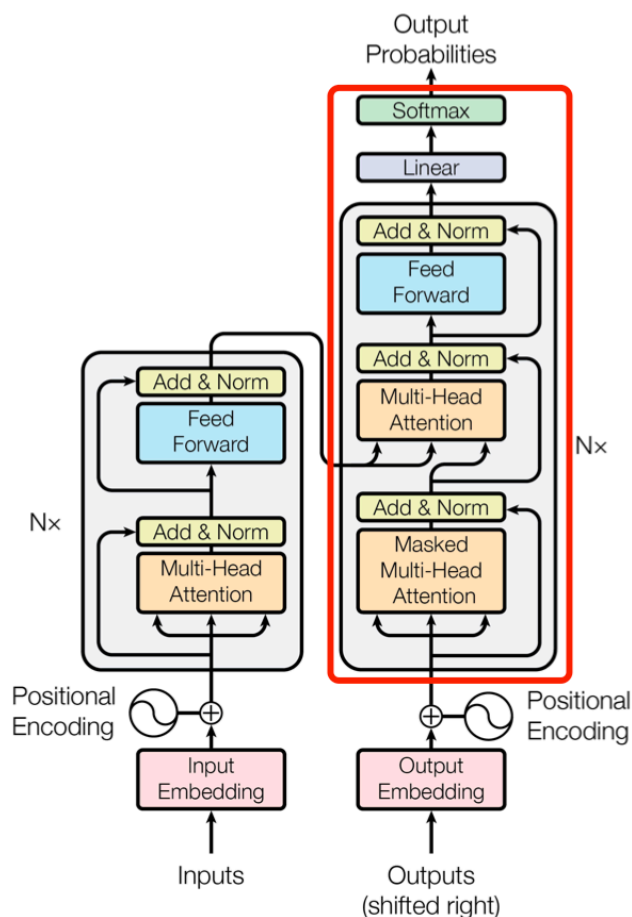
```
1 class EncoderLayer(nn.Module):
2     def __init__(self, d_model, ffn_hidden, n_head, drop_prob=0.1):
3         super(EncoderLayer, self).__init__()
4         self.mha = MultiHeadAttention(d_model, n_head)
5         self.ffn = PositionwiseFeedForward(d_model, ffn_hidden)
6         self.ln1 = LayerNorm(d_model)
7         self.ln2 = LayerNorm(d_model)
8         self.dropout1 = nn.Dropout(drop_prob)
9         self.dropout2 = nn.Dropout(drop_prob)
10
11     def forward(self, x, mask=None):
12         x_residual1 = x
13
14         # 1, compute multi-head attention
15         x = self.mha(q=x, k=x, v=x, mask=mask)
16
17         # 2, add residual connection and apply layer norm
18         x = self.ln1( x_residual1 + self.dropout1(x) )
19         x_residual2 = x
20
21         # 3, compute position-wise feed forward
22         x = self.ffn(x)
23
24         # 4, add residual connection and apply layer norm
25         x = self.ln2( x_residual2 + self.dropout2(x) )
26
27         return x
28
29 class Encoder(nn.Module):
30     def __init__(self, enc_voc_size, seq_len, d_model, ffn_hidden, n_head,
31                  n_layers, drop_prob=0.1, device='cpu'):
```

```

31     super().__init__()
32     self.emb = TransformerEmbedding(vocab_size = enc_voc_size,
33                                     max_len = seq_len,
34                                     d_model = d_model,
35                                     drop_prob = drop_prob,
36                                     device=device)
37     self.layers = nn.ModuleList([EncoderLayer(d_model, ffn_hidden, n_head,
drop_prob)
38                                     for _ in range(n_layers)])
39
40     def forward(self, x, mask=None):
41
42         x = self.emb(x)
43
44         for layer in self.layers:
45             x = layer(x, mask)
46         return x

```

五, Decoder 结构



上图右边红框框出来的是 Decoder block, Decoder 组件也是由 $N = 6$ 个相同的 Decoder block 堆叠而成。Decoder block 与 Encoder block 相似, 但是存在一些区别:

- 包含两个 Multi-Head Attention 层。
- 第一个 Multi-Head Attention 层采用了 Masked 操作。
- 第二个 Multi-Head Attention 层的 $\mathbf{K}$, $\mathbf{V}$ 矩阵使用 Encoder 的编码信息矩阵 $\mathbf{C}$ 进行计算, 而 $\mathbf{Q}$ 使用上一个

Decoder block 的输出计算。这样做的好处是在 Decoder 的时候，每一位单词都可以利用到 Encoder 所有单词的信息 (这些信息无需 **Mask**)

注意，解码器块中的第一个注意力层关联到解码器的所有（过去的）输入，但是第二注意力层使用编码器的输出。因此，它可以访问整个输入句子，以最好地预测当前单词。这是非常有用的，因为不同的语言可以有语法规则将单词按不同的顺序排列，或者句子后面提供的一些上下文可能有助于确定给定单词的最佳翻译。

另外，Decoder 组件后面还会接一个全连接层和 Softmax 层计算下一个翻译单词的概率。

Decoder 组件的代码实现如下所示：

```
1  class DecoderLayer(nn.Module):
2
3      def __init__(self, d_model, ffn_hidden, n_head, drop_prob):
4          super(DecoderLayer, self).__init__()
5          self.mha1 = MultiHeadAttention(d_model, n_head)
6          self.ln1 = LayerNorm(d_model)
7          self.dropout1 = nn.Dropout(p=drop_prob)
8
9          self.mha2 = MultiHeadAttention(d_model, n_head)
10         self.ln2 = LayerNorm(d_model)
11         self.dropout2 = nn.Dropout(p=drop_prob)
12
13         self.ffn = PositionwiseFeedForward(d_model, ffn_hidden)
14         self.ln3 = LayerNorm(d_model)
15         self.dropout3 = nn.Dropout(p=drop_prob)
16
17     def forward(self, dec_out, enc_out, trg_mask, src_mask):
18         x_residual1 = dec_out
19
20         # 1, compute multi-head attention
21         x = self.mha1(q=dec_out, k=dec_out, v=dec_out, mask=trg_mask)
22
23         # 2, add residual connection and apply layer norm
24         x = self.ln1( x_residual1 + self.dropout1(x) )
25
26         if enc_out is not None:
27             # 3, compute encoder - decoder attention
28             x_residual2 = x
29             x = self.mha2(q=x, k=enc_out, v=enc_out, mask=src_mask)
30
31             # 4, add residual connection and apply layer norm
32             x = self.ln2( x_residual2 + self.dropout2(x) )
33
34             # 5. positionwise feed forward network
35             x_residual3 = x
36             x = self.ffn(x)
37             # 6, add residual connection and apply layer norm
38             x = self.ln3( x_residual3 + self.dropout3(x) )
39
```

```

40         return x
41
42     class Decoder(nn.Module):
43         def __init__(self, dec_voc_size, max_len, d_model, ffn_hidden, n_head,
44             n_layers, drop_prob, device):
45             super().__init__()
46             self.emb = TransformerEmbedding(d_model=d_model,
47                 drop_prob=drop_prob,
48                 max_len=max_len,
49                 vocab_size=dec_voc_size,
50                 device=device)
51
52             self.layers = nn.ModuleList([DecoderLayer(d_model=d_model,
53                 ffn_hidden=ffn_hidden,
54                 n_head=n_head,
55                 drop_prob=drop_prob)
56                 for _ in range(n_layers)])
57
58             self.linear = nn.Linear(d_model, dec_voc_size)
59
60         def forward(self, trg, src, trg_mask, src_mask):
61             trg = self.emb(trg)
62
63             for layer in self.layers:
64                 trg = layer(trg, src, trg_mask, src_mask)
65
66             # pass to LM head
67             output = self.linear(trg)
68             return output

```

六，Transformer 总结

- Transformer 与 RNN 不同，可以比较好地并行训练。
- Transformer 本身是不能利用单词的顺序信息的，因此需要在输入中添加位置 Embedding，否则 Transformer 就是一个词袋模型了。
- Transformer 的重点是 Self-Attention 结构，其中用到的 **Q, K, V** 矩阵通过输出进行线性变换得到。
- Transformer 中 Multi-Head Attention 中有多个 Self-Attention，可以捕获单词之间多种维度上的相关系数 attention score。

6.1，Transformer 完整代码实现

基于前面实现的 Encoder 和 Decoder 组件，我们可以实现 Transformer 模型的完整代码，如下所示：

```

1  import torch
2  from torch import nn
3
4  from models.model.decoder import Decoder
5  from models.model.encoder import Encoder

```

```

6
7
8 class Transformer(nn.Module):
9
10     def __init__(self, src_pad_idx, trg_pad_idx, trg_sos_idx, enc_voc_size,
11 dec_voc_size, d_model, n_head, max_len,
12                 ffn_hidden, n_layers, drop_prob, device):
13         super().__init__()
14         self.src_pad_idx = src_pad_idx
15         self.trg_pad_idx = trg_pad_idx
16         self.trg_sos_idx = trg_sos_idx
17         self.device = device
18         self.encoder = Encoder(d_model=d_model,
19                                n_head=n_head,
20                                max_len=max_len,
21                                ffn_hidden=ffn_hidden,
22                                enc_voc_size=enc_voc_size,
23                                drop_prob=drop_prob,
24                                n_layers=n_layers,
25                                device=device)
26
27         self.decoder = Decoder(d_model=d_model,
28                                n_head=n_head,
29                                max_len=max_len,
30                                ffn_hidden=ffn_hidden,
31                                dec_voc_size=dec_voc_size,
32                                drop_prob=drop_prob,
33                                n_layers=n_layers,
34                                device=device)
35
36     def forward(self, src, trg):
37         src_mask = self.make_pad_mask(src, src, self.src_pad_idx, self.src_pad_idx)
38
39         src_trg_mask = self.make_pad_mask(trg, src, self.trg_pad_idx,
40 self.src_pad_idx)
41
42         trg_mask = self.make_pad_mask(trg, trg, self.trg_pad_idx, self.trg_pad_idx)
43
44         * \
45             self.make_no_peak_mask(trg, trg)
46
47         enc_src = self.encoder(src, src_mask)
48         output = self.decoder(trg, enc_src, trg_mask, src_trg_mask)
49         return output
50
51     def make_pad_mask(self, q, k, q_pad_idx, k_pad_idx):
52         len_q, len_k = q.size(1), k.size(1)
53
54         # batch_size x 1 x 1 x len_k
55         k = k.ne(k_pad_idx).unsqueeze(1).unsqueeze(2)

```

```

52         # batch_size x 1 x len_q x len_k
53         k = k.repeat(1, 1, len_q, 1)
54
55         # batch_size x 1 x len_q x 1
56         q = q.ne(q_pad_idx).unsqueeze(1).unsqueeze(3)
57         # batch_size x 1 x len_q x len_k
58         q = q.repeat(1, 1, 1, len_k)
59
60         mask = k & q
61         return mask
62
63     def make_no_peak_mask(self, q, k):
64         len_q, len_k = q.size(1), k.size(1)
65
66         # len_q x len_k
67         mask = torch.tril(torch.ones(len_q,
len_k)).type(torch.BoolTensor).to(self.device)
68
69         return mask

```

参考资料

1. <https://github.com/hyunwoongko/transformer>
2. [The Illustrated Transformer](#)
3. [Learning Word Embedding](#)
4. [Transformer模型详解（图解最完整版）](#)